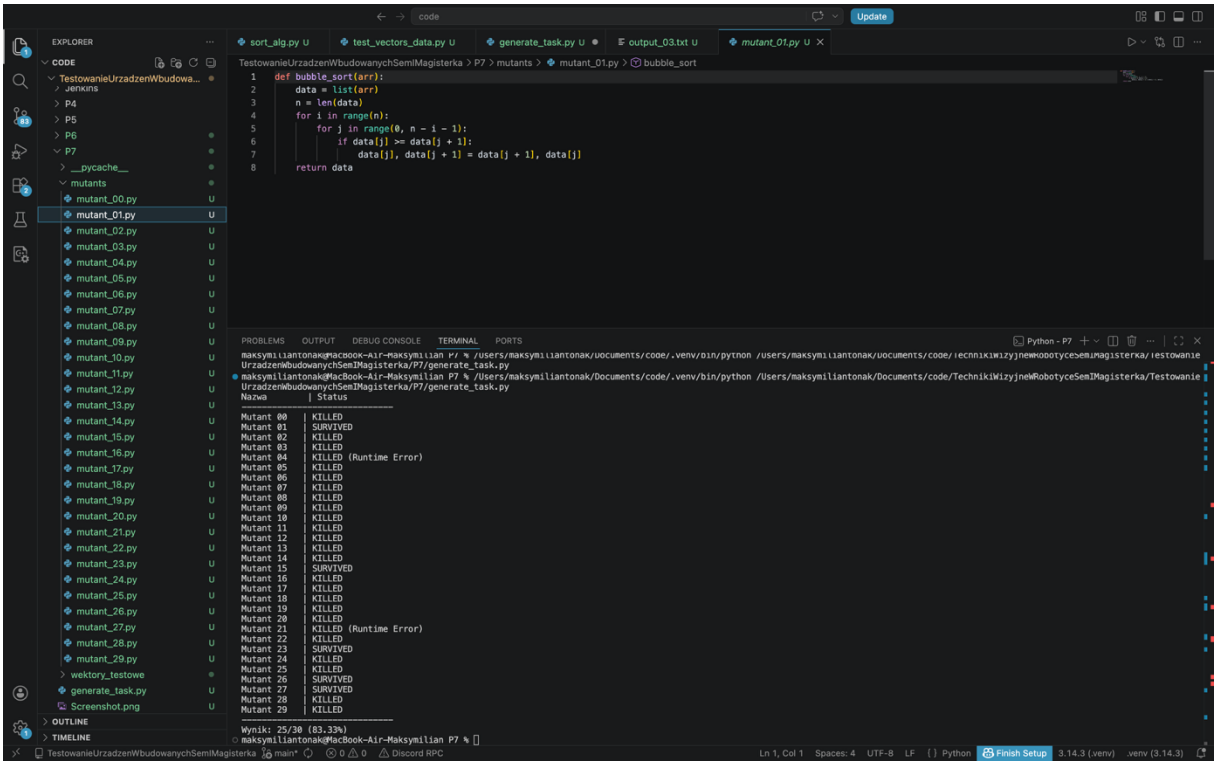
Projekt został podzielony na trzy współpracujące ze sobą pliki. Plik sort_alg.py zawiera czysty kod źródłowy algorytmu Bubble Sort, który służy jako wzorzec do generowania późniejszych mutacji. Następny plik pełni rolę bazy danych testowych, przechowując 20 zróżnicowanych zestawów wejść i wyjść. Ostatni element, generate_task.py, to główny silnik programu, który wczytuje kod źródłowy, automatycznie tworzy 30 wariantów z błędami (mutantów), generuje 40 plików tekstowych z wektorami oraz przeprowadza testy dla każdego przypadku.



Program w pierwszej fazie pobiera 20 zestawów danych testowych z zewnętrznego pliku oraz wczytuje kod źródłowy algorytmu sortowania jako ciąg znaków, a następnie automatycznie generuje 40 plików tekstowych zawierających pary wejść i oczekiwanych wyjść. Kolejnym krokiem jest proces mutacji, w którym skrypt tworzy 30 wariantów kodu poprzez tekstową podmianę operatorów logicznych i arytmetycznych, zapisując każdą zepsutą wersję w osobnym pliku. W fazie końcowej skrypt dynamicznie uruchamia każdego mutanta w izolowanym środowisku i konfrontuje go ze wszystkimi zapisanymi wcześniej wektorami testowymi. Poprzez porównanie wyniku zwróconego przez zmodyfikowany kod z poprawnym wzorcem, program klasyfikuje błędy jako wykryte lub przeoczone i wyświetla wynik działania.

Przyczyną przetrwania 5 z 30 mutantów jest fakt, iż zmienia się zapis kodu, ale nie modyfikuje się jego zachowanie dla konkretnych zestawów danych. Zamiana operatora większości na operator większy-równy nie wpływa na ostateczną kolejność liczb, jeśli w testowanej tablicy nie występują obok siebie identyczne wartości, przez co test weryfikujący jedynie wynik końcowy nie jest w stanie wyłapać takiej zmiany.

Kody źródłowe:

sort_alg.py

```
def bubble_sort(arr):
    data = list(arr)
    n = len(data)
    for i in range(n):
        for j in range(0, n - i - 1):
            if data[j] > data[j + 1]:
                data[j], data[j + 1] = data[j + 1], data[j]
    return data
```

test_vectors_data.py

```
# Format: (wejście, oczekiwany_wynik)
TEST_DATA = [
    ([3, 1, 4, 2], [1, 2, 3, 4]), ([1, 2], [1, 2]), ([2, 1], [1, 2]),
    ([4, 3, 2, 1], [1, 2, 3, 4]), ([1, 1, 1], [1, 1, 1]), ([], []),
    ([5], [5]), ([10, -5, 0, 2], [-5, 0, 2, 10]), ([9, 8, 7, 6, 5], [5, 6, 7, 8,
9]),
    ([1, 2, 3, 4, 5], [1, 2, 3, 4, 5]), ([0, 0, 0, 1], [0, 0, 0, 1]),
    ([100, 10, 50], [10, 50, 100]), ([2, 1, 2, 1], [1, 1, 2, 2]),
    ([0.5, 0.1, 0.9], [0.1, 0.5, 0.9]), ([10, 9, 8, 7, 6, 5, 4], [4, 5, 6, 7, 8,
9, 10]),
    ([1, 5, 2, 4, 3], [1, 2, 3, 4, 5]), ([7, 7, 2, 2], [2, 2, 7, 7]),
    ([-1, -2, -3], [-3, -2, -1]), ([10, 20, 30], [10, 20, 30]), ([5, 4, 3, 2],
[2, 3, 4, 5])
]
```

generate_task.py

```
import os
import shutil

MUTATIONS = [
    ("if data[j] > data[j + 1]:", "if data[j] < data[j + 1]:"),
    ("if data[j] > data[j + 1]:", "if data[j] >= data[j + 1]:"),
    ("if data[j] > data[j + 1]:", "if data[j] <= data[j + 1]:"),
    ("if data[j] > data[j + 1]:", "if True:"),
    ("range(0, n - i - 1)", "range(0, n - i)"),
    ("range(0, n - i - 1)", "range(0, 0)"),
    ("data[j], data[j + 1] = data[j + 1], data[j]", "pass"),
    ("n = len(data)", "n = len(data) - 1"),
    ("n = len(data)", "n = 0"),
    ("return data", "return []"),
    ("if data[j] > data[j + 1]:", "if data[j] != data[j + 1]:"),
    ("for i in range(n):", "for i in range(1):"),
    ("data = list(arr)", "data = []"),
    ("data[j], data[j + 1] = data[j + 1], data[j]", "data[j] = data[j+1]"),
```

```
    ("data[j], data[j + 1] = data[j + 1], data[j]", "data[j+1] = data[j]"),
    ("range(n)", "range(n-1)"),
    ("if data[j] > data[j + 1]:", "if data[j] == data[j + 1]:"),
    ("j + 1", "j"),
    ("j + 1", "j - 1"),
    ("data[j]", "data[0]"),
    ("range(0, n - i - 1)", "range(1, n - i - 1)"),
    ("i", "0"),
    ("range(n)", "range(n // 2)"),
    ("data = list(arr)", "data = arr"),
    ("if data[j] > data[j + 1]:", "if data[j+1] > data[j]:"),
    ("data[j] > data[j + 1]", "data[j] > data[0]"),
    ("n - i - 1", "n - 1"),
    ("range(n)", "range(n + 1)"),
    ("data[j], data[j + 1]", "data[j], data[j]"),
    ("return data", "return arr[::-1]")
]

def generate():
    from TestowanieUrzadzenWbudowanychSemIMagisterka.P7.test_vectors_data import
TEST_DATA

    # 1. Tworzenie wektorów (pliki tekstowe)
    v_folder = "wektory_testowe"
    if os.path.exists(v_folder): shutil.rmtree(v_folder)
    os.makedirs(v_folder)
    for i, (inp, exp) in enumerate(TEST_DATA):
        with open(f"{v_folder}/input_{i:02d}.txt", "w") as f: f.write(str(inp))
        with open(f"{v_folder}/output_{i:02d}.txt", "w") as f: f.write(str(exp))

    # 2. Tworzenie mutantów (pliki .py)
    m_folder = "mutants"
    if os.path.exists(m_folder): shutil.rmtree(m_folder)
    os.makedirs(m_folder)
    with open("sort_alg.py", "r") as f: original = f.read()

    print(f"{'Nazwa':<12} | {'Status'}")
    print("-" * 30)
    killed = 0
    for i, (old, new) in enumerate(MUTATIONS):
        mutant_code = original.replace(old, new, 1)
        name = f"mutant_{i:02d}.py"
        with open(f"{m_folder}/{name}", "w") as f: f.write(mutant_code)

        # Testowanie mutanta
        try:
            namespace = {}
            exec(mutant_code, namespace)
            func = namespace['bubble_sort']
            is_killed = False
            for inp, exp in TEST_DATA:
```

```
                if func(list(inp)) != exp:
                    is_killed = True
                    break
            status = "KILLED" if is_killed else "SURVIVED"
        except:
            status = "KILLED (Runtime Error)"

        if "KILLED" in status: killed += 1
        print(f"Mutant {i:02d}   | {status}")

    print("-" * 30)
    print(f"Wynik: {killed}/{len(MUTATIONS)}
({(killed/len(MUTATIONS))*100:.2f}%)")

if __name__ == "__main__":
    generate()
```